

# Guia de Estudos: Primeiros Passos com UIKit para Desenvolvimento iOS (Nível Iniciante)

Sejam bem-vindos à jornada de criação de aplicativos para iPhone e iPad utilizando o **UIKit**, o framework fundamental da Apple para construir interfaces de usuário (UI) por muitos anos. Pensem neste guia como o "mapa da mina" inicial para desbravar esse território. Nosso foco será o primeiro contato, entendendo as peças básicas e como elas se encaixam.

**Objetivo:** Ao final deste guia, você será capaz de criar um projeto iOS, adicionar e configurar componentes básicos da interface usando o Interface Builder (Storyboard) e conectar esses elementos ao código Swift para criar interações simples.

## Pré-requisitos:

1. **Hardware:** Um computador Mac com uma versão recente do macOS.
2. **Software:** Xcode instalado (disponível gratuitamente na Mac App Store).
3. **Conhecimento:**
  - Noções básicas de programação (o que são variáveis, tipos de dados, condicionais if/else, loops for/while, funções/métodos).
  - Familiaridade básica com a linguagem **Swift** é altamente recomendada. Se não tiver, dedique um tempo inicial para aprender a sintaxe básica de Swift.
4. **Atitude:** Curiosidade, paciência e vontade de experimentar (e errar!).

---

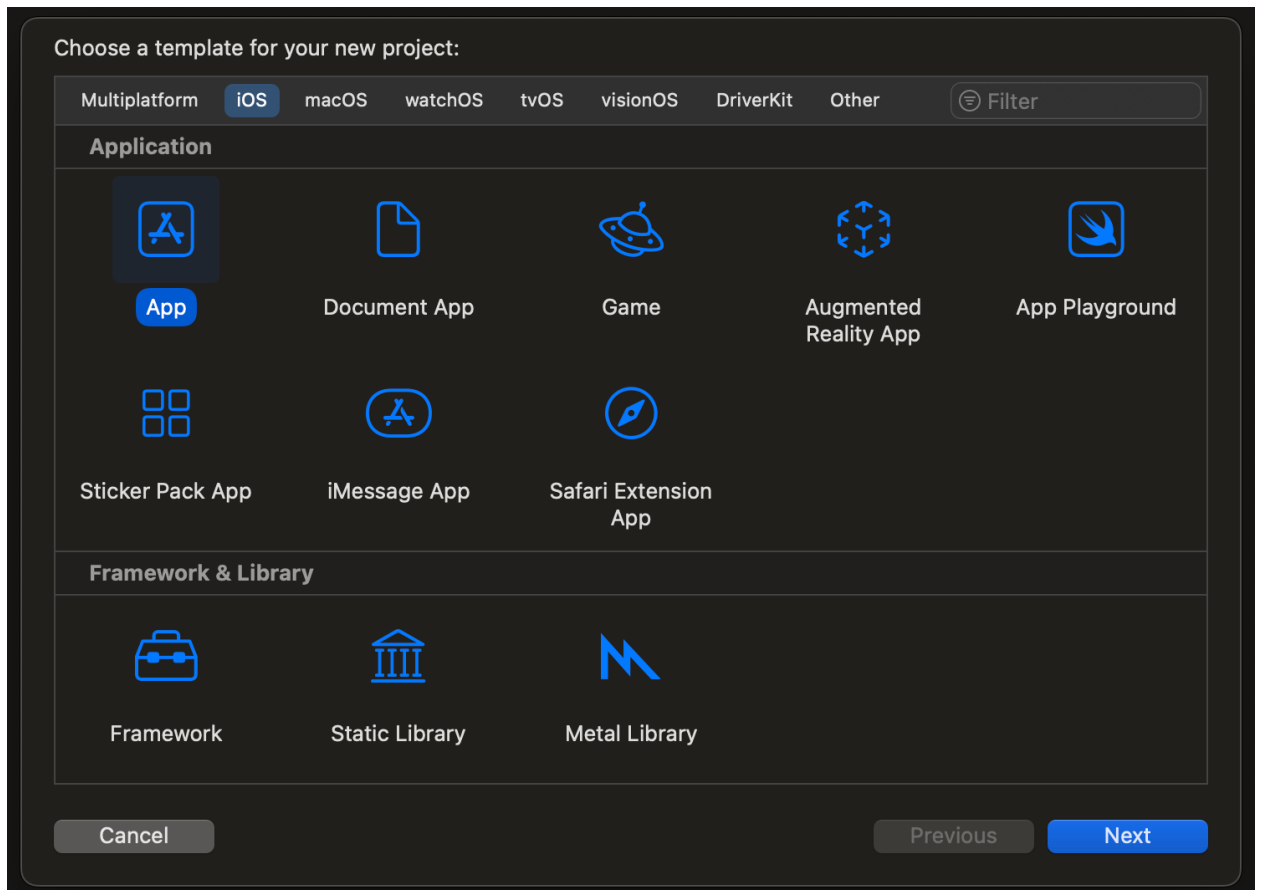
## Passo a Passo:

### Passo 1: Conhecendo o Ambiente - Xcode e a Criação do Projeto

- **O que é Xcode?** É o Ambiente de Desenvolvimento Integrado (IDE) da Apple. Nele, você escreverá código, desenhará interfaces, testará e depurará seus apps.
- **Atividade:**
  1. Abra o Xcode.
  2. Selecione "Create a new Xcode project".

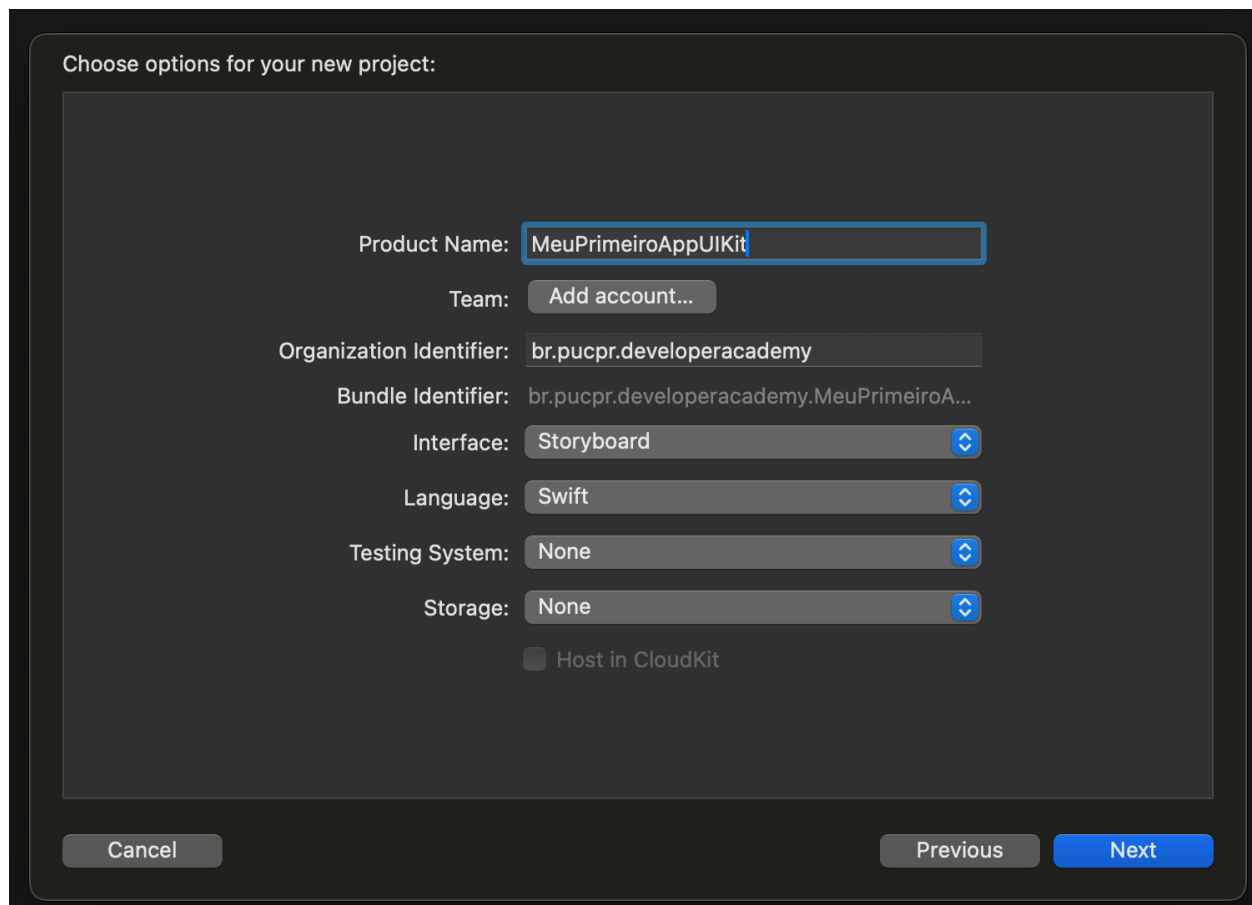


3. Na aba "iOS", escolha o template "App" e clique em "Next".



4. Preencha as opções:

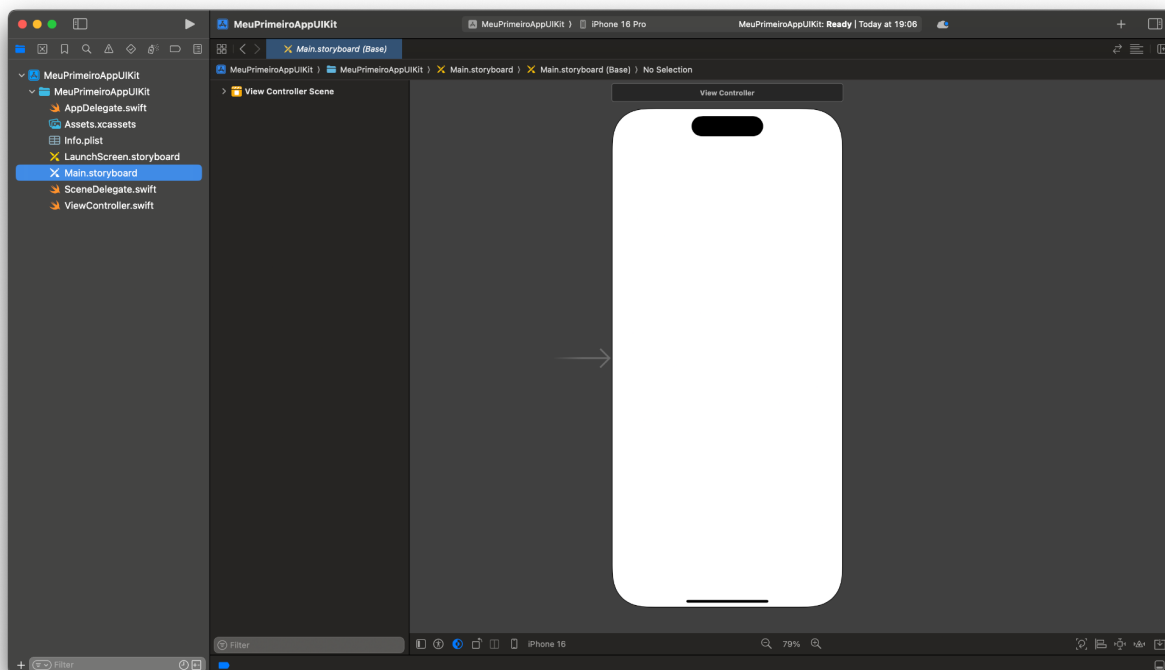
- **Product Name:** O nome do seu app (ex: MeuPrimeiroAppUIKit).
- **Team:** Pode deixar sem.
- **Organization Identifier:** Um identificador único, geralmente no formato reverso do domínio (ex: com.seunome). Juntos, formam o *Bundle Identifier*.
- **Interface:** Selecione "**Storyboard**". (UIKit oferece Storyboards e SwiftUI para design visual, e também criação programática. Começaremos pelo visual).
- **Language:** Selecione "**Swift**".
- **Testing System:** Selecione "**None**".
- **Storage:** Selecione "**None**".



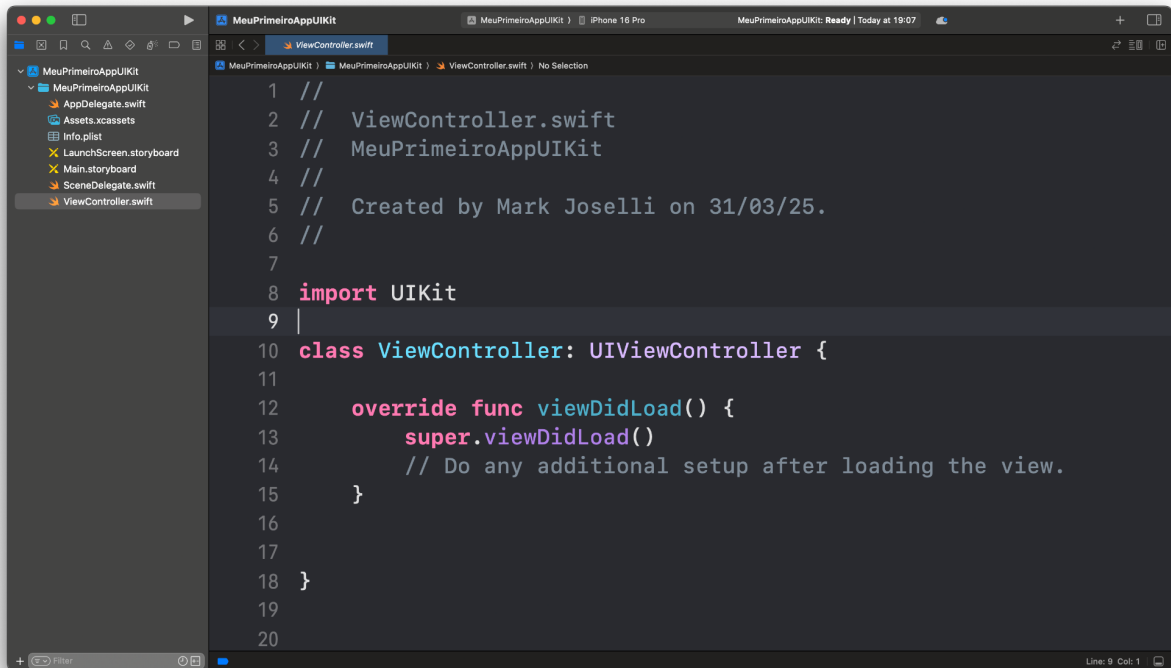
5. Clique em "Next" e escolha onde salvar seu projeto. Você pode criar o repositório local git case queira (recomendado).
- **Explorando:** Dedique alguns minutos para se familiarizar com as áreas principais do Xcode: Navegador de Projeto (esquerda), Editor Principal (centro), Inspetores (direita), Área de Debug (embaixo) e a Biblioteca de Objetos (canto inferior direito ou via menu View > Show Library).

## Passo 2: O Coração da Tela - UIViewController e Storyboard

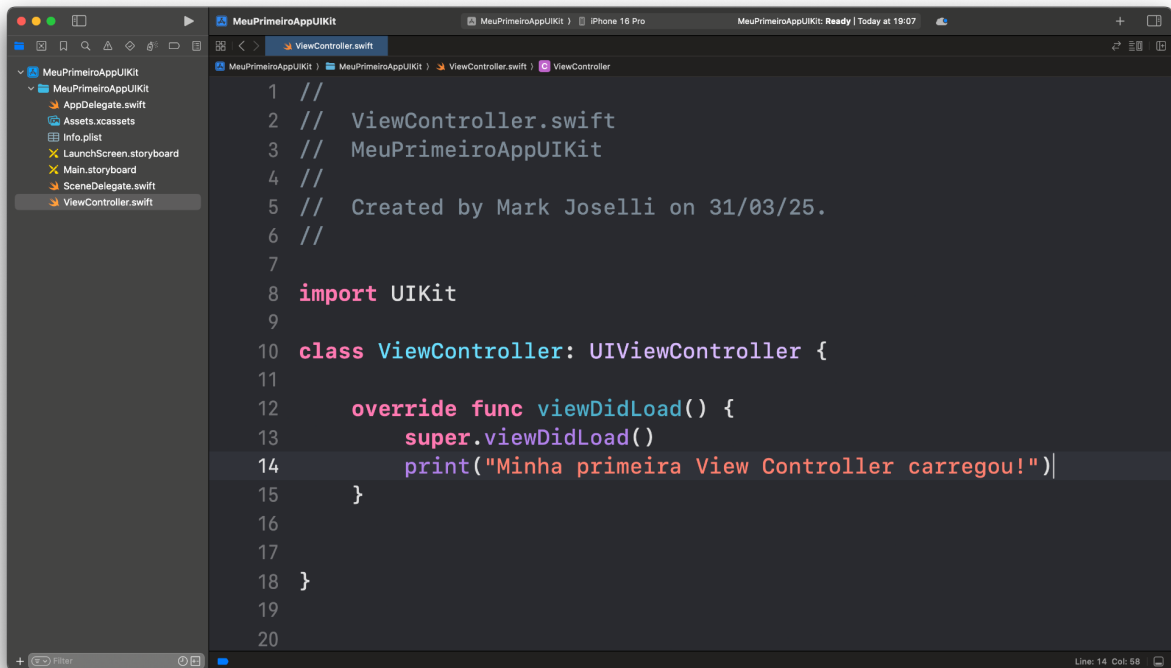
- **O que é UIViewController?** Pense nele como o "cérebro" de uma tela. Ele gerencia a *view* (a parte visual) e responde a interações do usuário. Cada tela ou seção principal do seu app geralmente terá um UIViewController associado.
- **O que é Storyboard?** É um arquivo (.storyboard) onde você pode visualizar e desenhar as telas (View Controllers) e as transições (Segues) entre elas.
- **Atividade:**
  1. No Navegador de Projeto, encontre e clique no arquivo Main.storyboard. Você verá uma representação visual de uma tela em branco – essa é a *view* gerenciada pelo seu ViewController inicial.



2. Clique no arquivo ViewController.swift. Este é o código Swift associado àquela tela no Storyboard. Observe a função viewDidLoad(). Este método é chamado *uma vez* quando a view do controller é carregada na memória – um ótimo lugar para configurações iniciais.

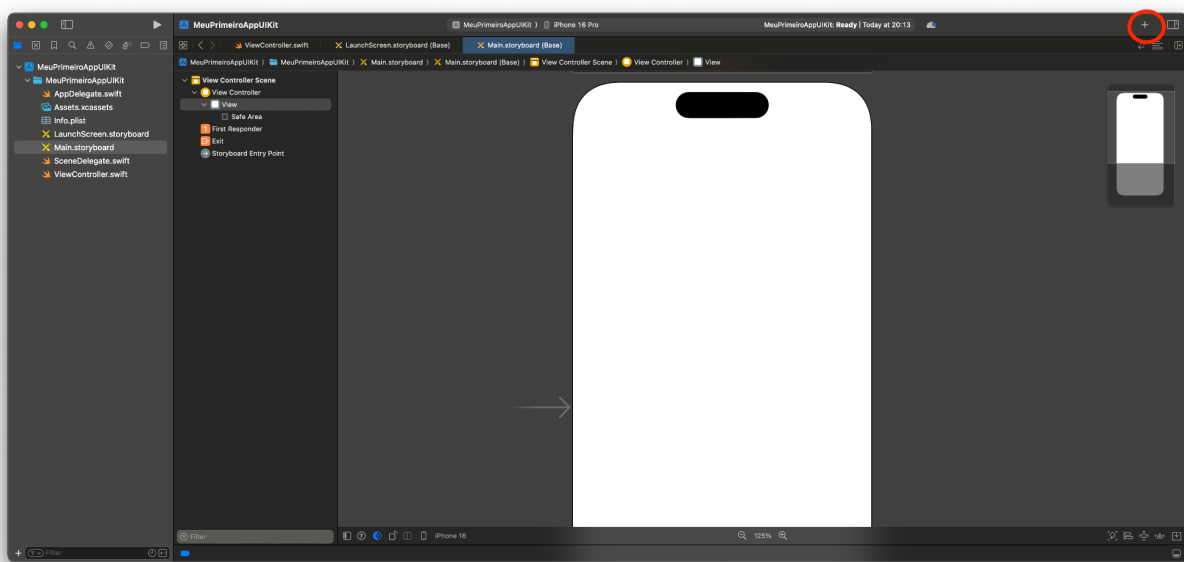


3. Adicione a linha `print("Minha primeira View Controller carregou!")` dentro das chaves `{}` da função `viewDidLoad()`.

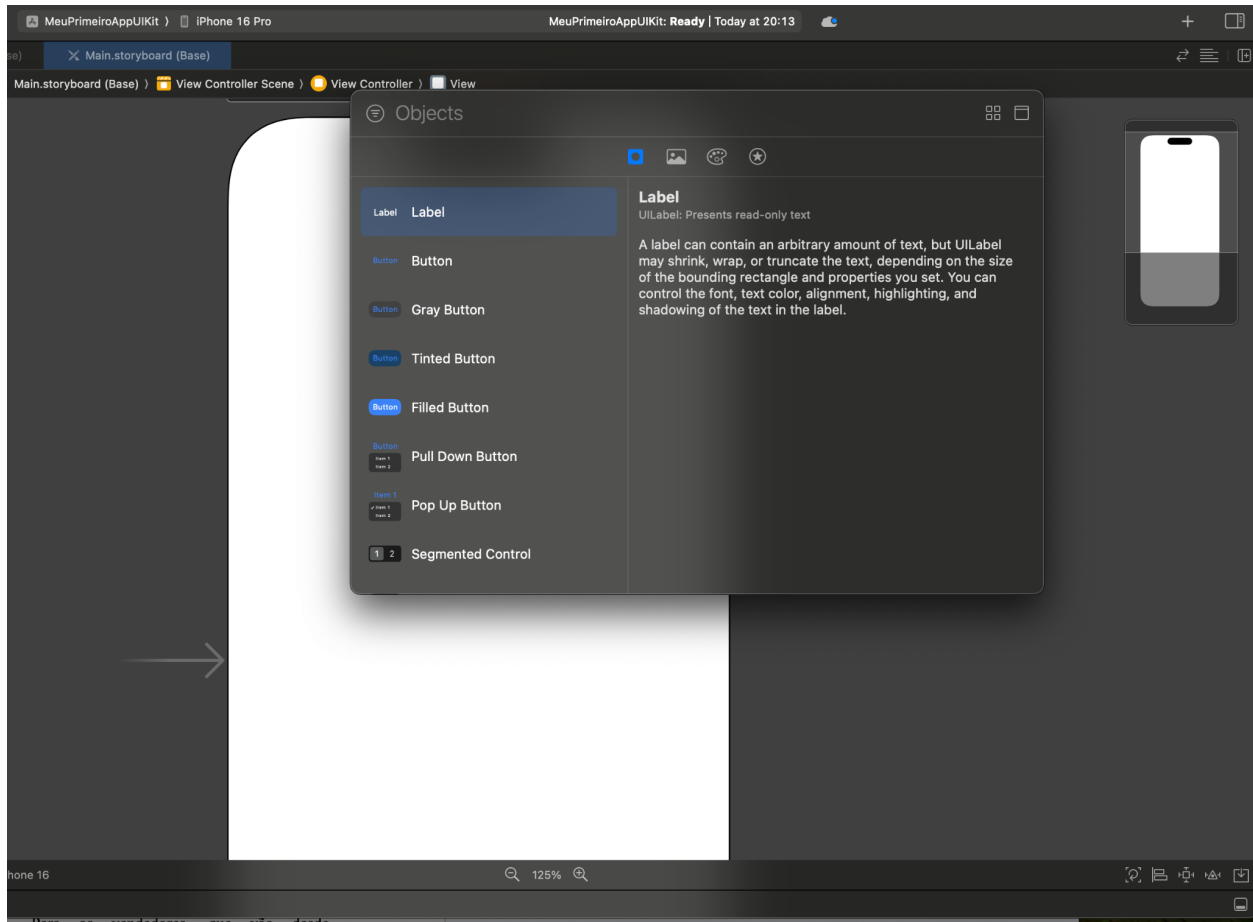


### Passo 3: Adicionando Peças Visuais - Componentes Básicos do UIKit

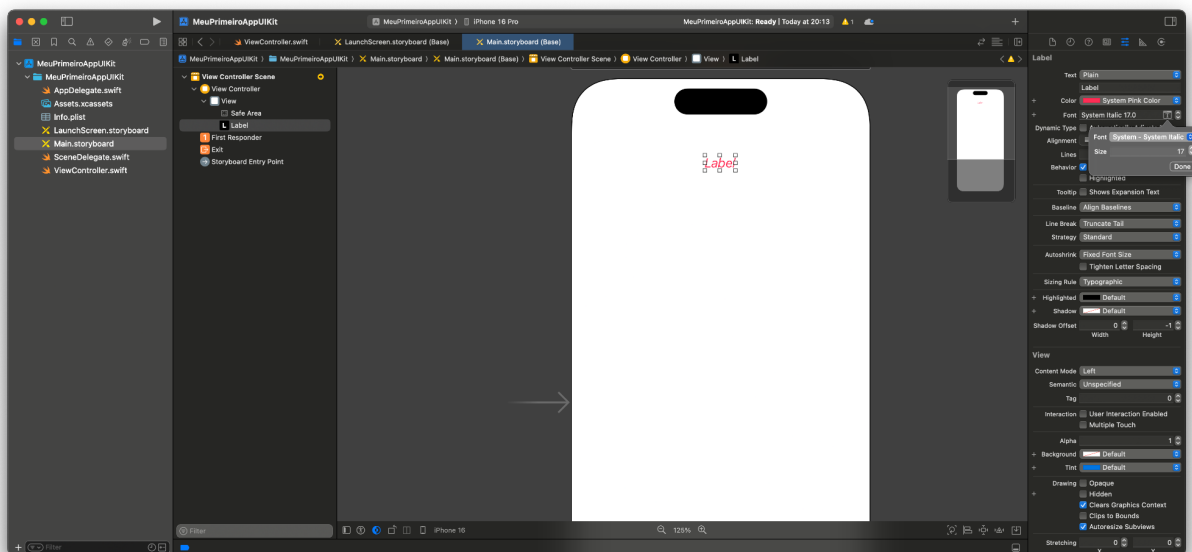
- **A Biblioteca de Objetos:** É onde você encontra todos os elementos visuais (e outros objetos) que pode usar.
- **Componentes Essenciais (Nossa Caixa de Ferramentas Inicial):**
  - UIView: O bloco de construção fundamental. Tudo que você vê na tela é, ou está contido dentro de, uma UIView. Pense nela como uma tela em branco retangular.
  - UILabel: Usado para exibir texto estático ou dinâmico.
  - UIButton: Permite que o usuário realize ações através de toques.
  - UIImageView: Usado para exibir imagens.
  - UITextField: Permite que o usuário insira uma linha de texto.
- **Atividade:**
  1. Volte para o Main.storyboard.
  2. Abra a Biblioteca de Objetos (ícone "+" no menu superior direito ou com o atalho: Shift + Command + L).



3. Arraste e solte um UILabel para dentro da view no Storyboard.



4. Com o Label selecionado, vá até o **Inspetor de Atributos** (ícone de controle deslizante na barra direita) e mude o texto padrão para "Olá, Mundo!". Experimente mudar a fonte e a cor.





5. Arraste e solte um UITextField. No Inspetor de Atributos, defina um "Placeholder" (texto de dica) como "Digite seu nome".
6. Arraste e solte um UIButton. No Inspetor de Atributos, mude o título para "Clique Aqui".

ryboard (Base)

e) > View Controller Scene > View Controller > View > B Clique Aqui

*Olá Mundo!*

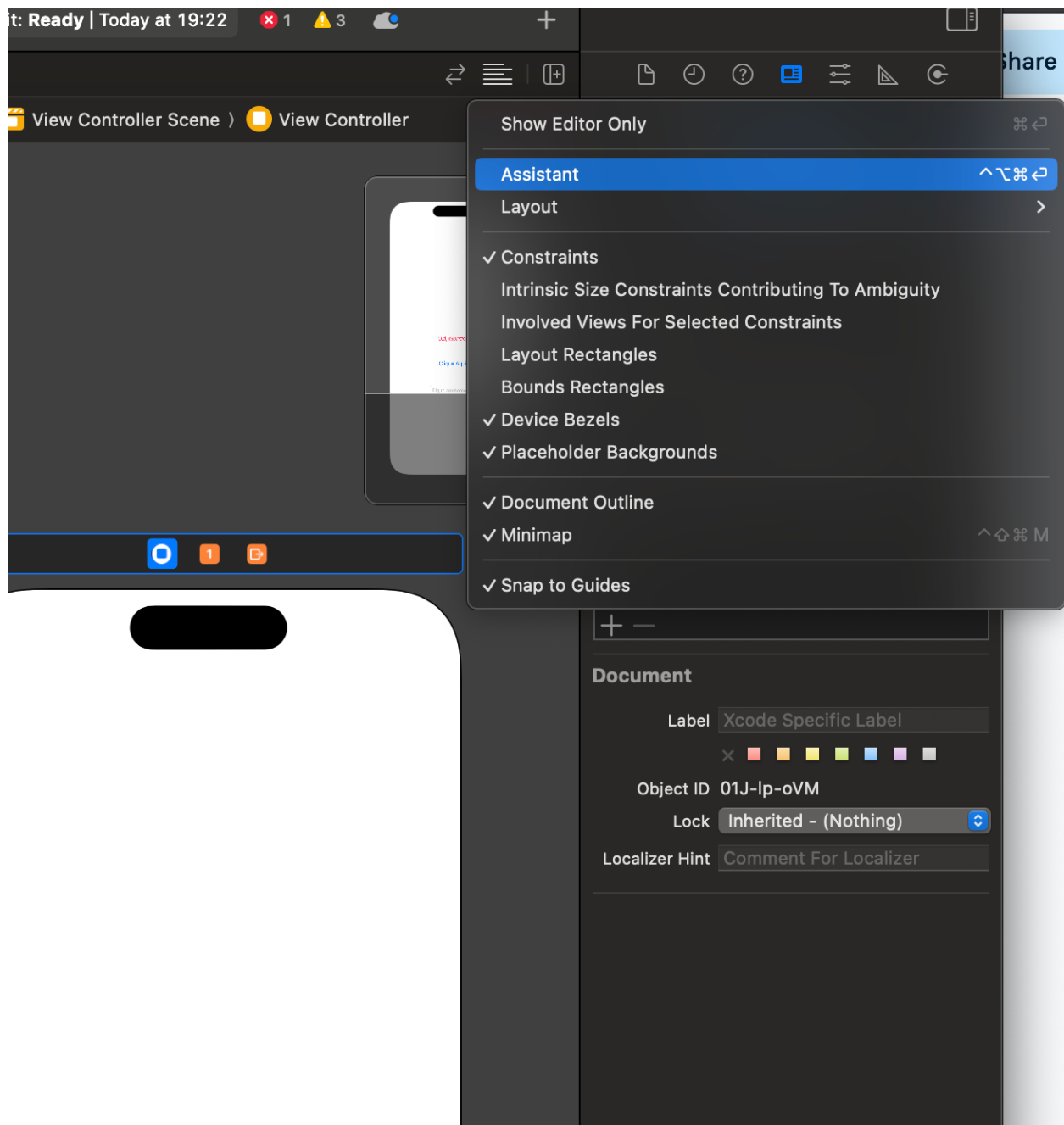
Digite seu...

Clique Aqui

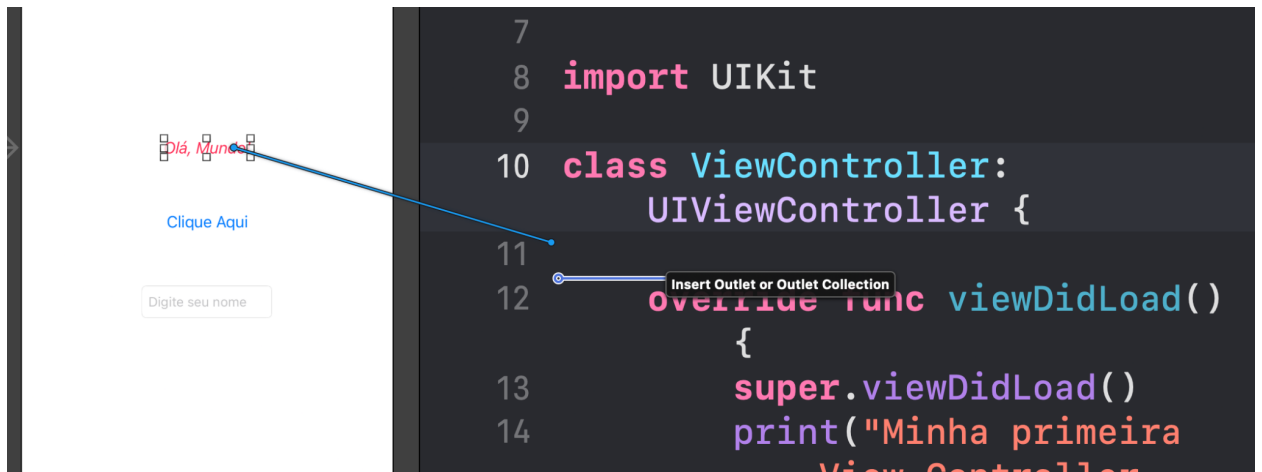
125%

#### Passo 4: Conectando o Visual ao Código - @IBOutlet e @IBAction

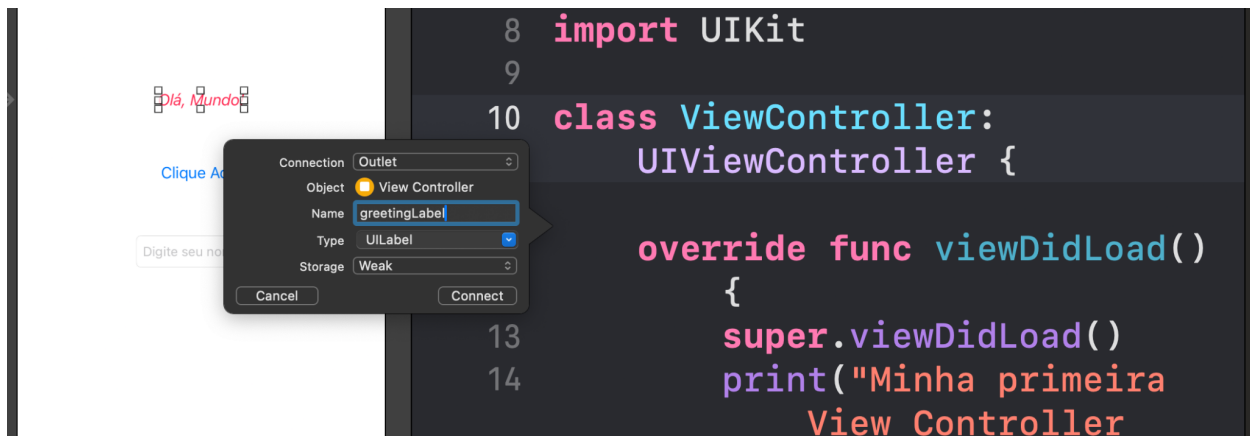
- **A Mágica da Conexão:** Para que seu código Swift possa interagir com os elementos visuais (ler o texto de um UITextField, mudar o texto de um UILabel, saber quando um UIButton foi tocado), precisamos criar conexões.
  - @IBOutlet: Cria uma *referência* (uma variável) no seu código Swift para um elemento específico no Storyboard.
  - @IBAction: Cria um *método* (uma função) no seu código Swift que será executado quando uma determinada *ação* ocorrer em um elemento do Storyboard (como o toque em um botão).
- **Atividade (Usando o Assistant Editor):**
  1. Com o Main.storyboard aberto, clique no ícone "Adjust Editor Options" (ícone de lista) no canto superior direito do Xcode e selecione "Assistant". Isso dividirá a tela, mostrando o Storyboard de um lado e (idealmente) o ViewController.swift do outro. Se não mostrar o ViewController.swift, selecione-o manualmente na barra de navegação superior do editor assistente.



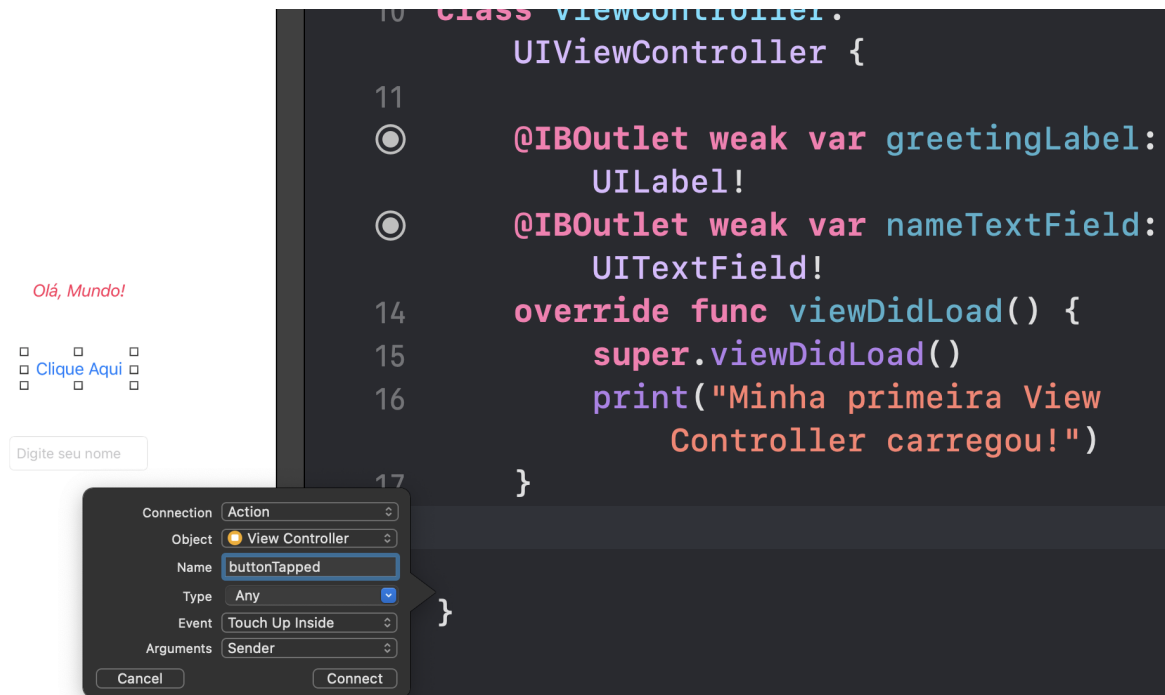
2. **Criando um Outlet para o Label:** Segure a tecla Control, clique no UILabel no Storyboard e arraste o cursor para dentro da classe ViewController no código Swift (logo abaixo da linha `class ViewController: UIViewController {}`).



3. Solte o mouse, uma pequena janela aparecerá.
  - Garanta que "Connection" esteja como "Outlet".
  - Dê um nome significativo (camelCase), como greetingLabel.
  - Clique em "Connect". Você verá uma linha @IBOutlet weak var greetingLabel: UILabel! no seu código.



4. **Criando um Outlet para o TextField:** Repita o processo para o UITextField, nomeando o Outlet como nameTextField.
5. **Criando uma Action para o Button:** Segure a tecla Control, clique no UIButton no Storyboard e arraste para dentro da classe ViewController, *abaixo* da função viewDidLoad. Solte o mouse.
  - Mude "Connection" para "Action".
  - Dê um nome significativo para a função, como buttonTapped.
  - Garanta que "Type" esteja como UIButton.
  - Clique em "Connect". Você verá uma função @IBAction func buttonTapped(\_ sender: Any) { ... } no seu código.



6. **Adicionando Interatividade:** Dentro da função `buttonTapped`, adicione o seguinte código:

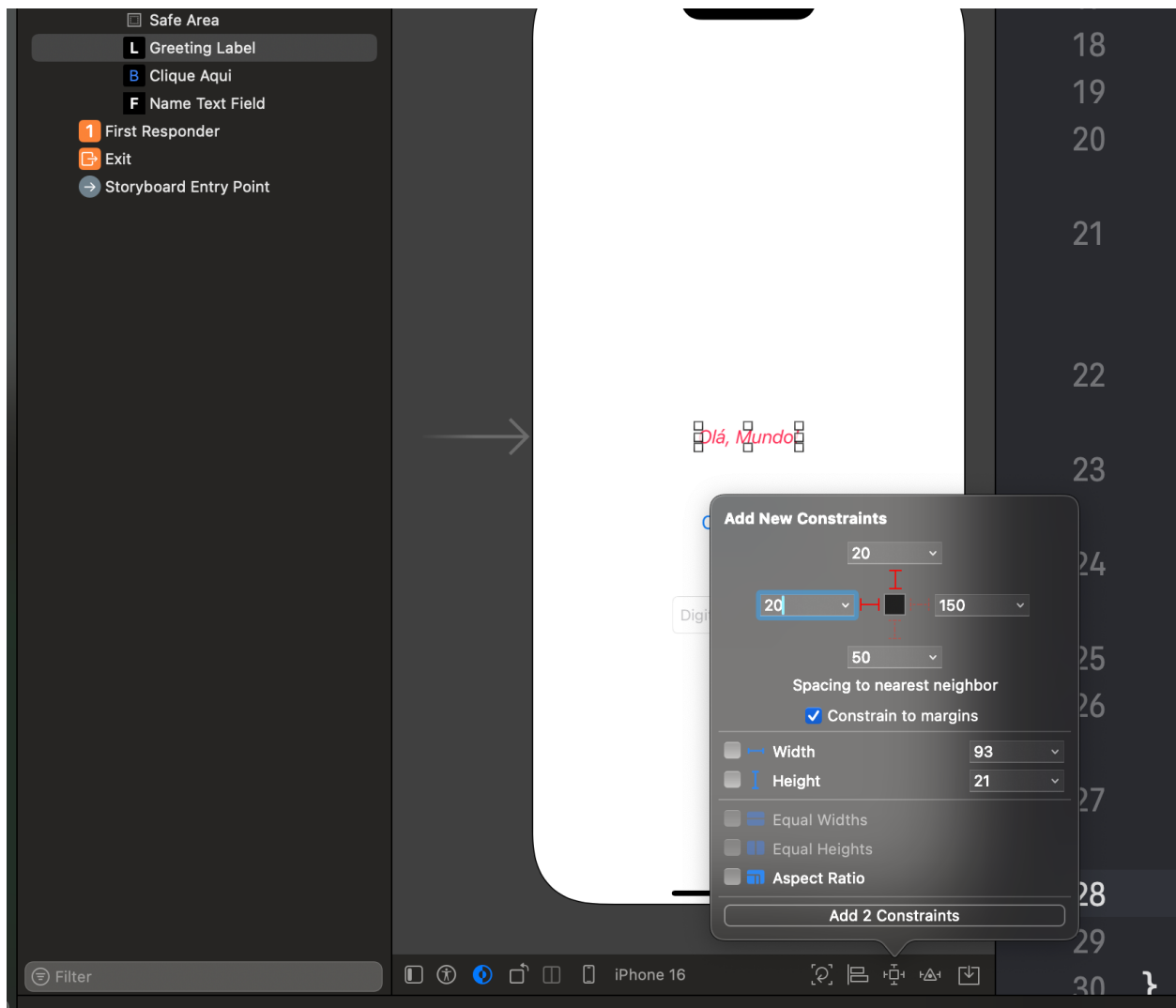
```
// Pega o texto do TextField (se houver), senão usa "Mundo"
let name = nameTextField.text ?? "Mundo"
// Garante que o nome não esteja vazio
let finalName = name.isEmpty ? "Mundo" : name
// Atualiza o texto do Label
greetingLabel.text = "Olá, \ \(finalName)!"
print("Botão clicado! Nome: \ \(finalName)")
```



### Passo 5: Organizando a Interface - Auto Layout (Introdução)

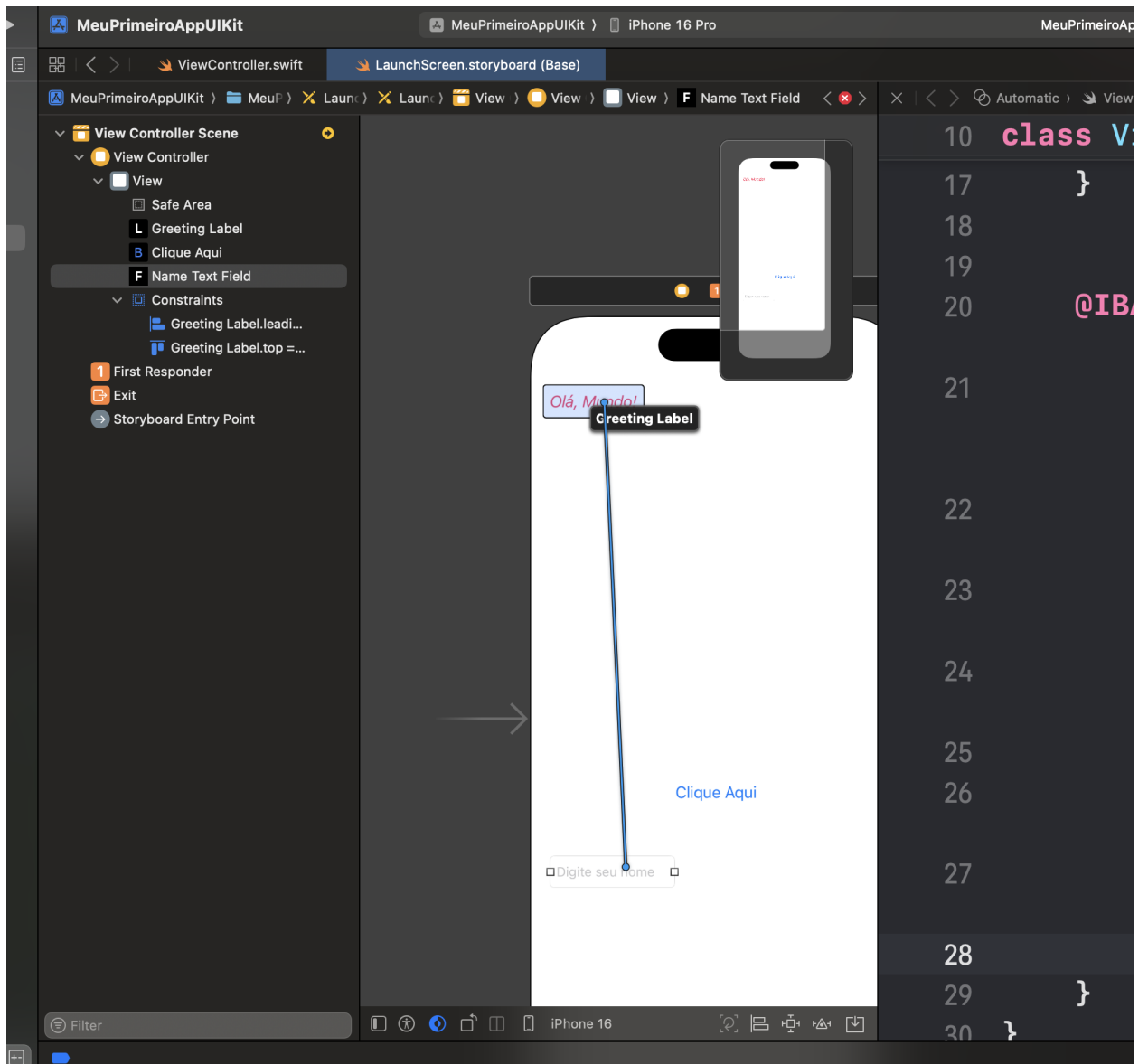
- **O Problema:** Dispositivos iOS têm diferentes tamanhos de tela e orientações (retrato/paisagem). Como garantir que sua interface fique boa em todos eles?
- **A Solução (Básica): Auto Layout.** Você define *regras* (constraints) que descrevem como os elementos devem se posicionar e se dimensionar em relação à tela ou a outros elementos. Cada elemento deve ter pelo menos duas constraints, uma para definir sua posição horizontalmente e outra para verticalmente.
- **Atividade (Conceito Inicial):**
  1. Selecione o UILabel no Storyboard.
  2. Na parte inferior direita da tela do Storyboard, encontre os ícones de Auto Layout. Clique no ícone "Add New Constraints" (parece um Tie Fighter ou um quadrado com pinos).
  3. Marque as caixas de constraint para a **borda superior** e a **borda inicial (esquerda)**. Defina valores (ex: 20 pontos do topo, 20 pontos da esquerda). Clique em "Add 2 Constraints".

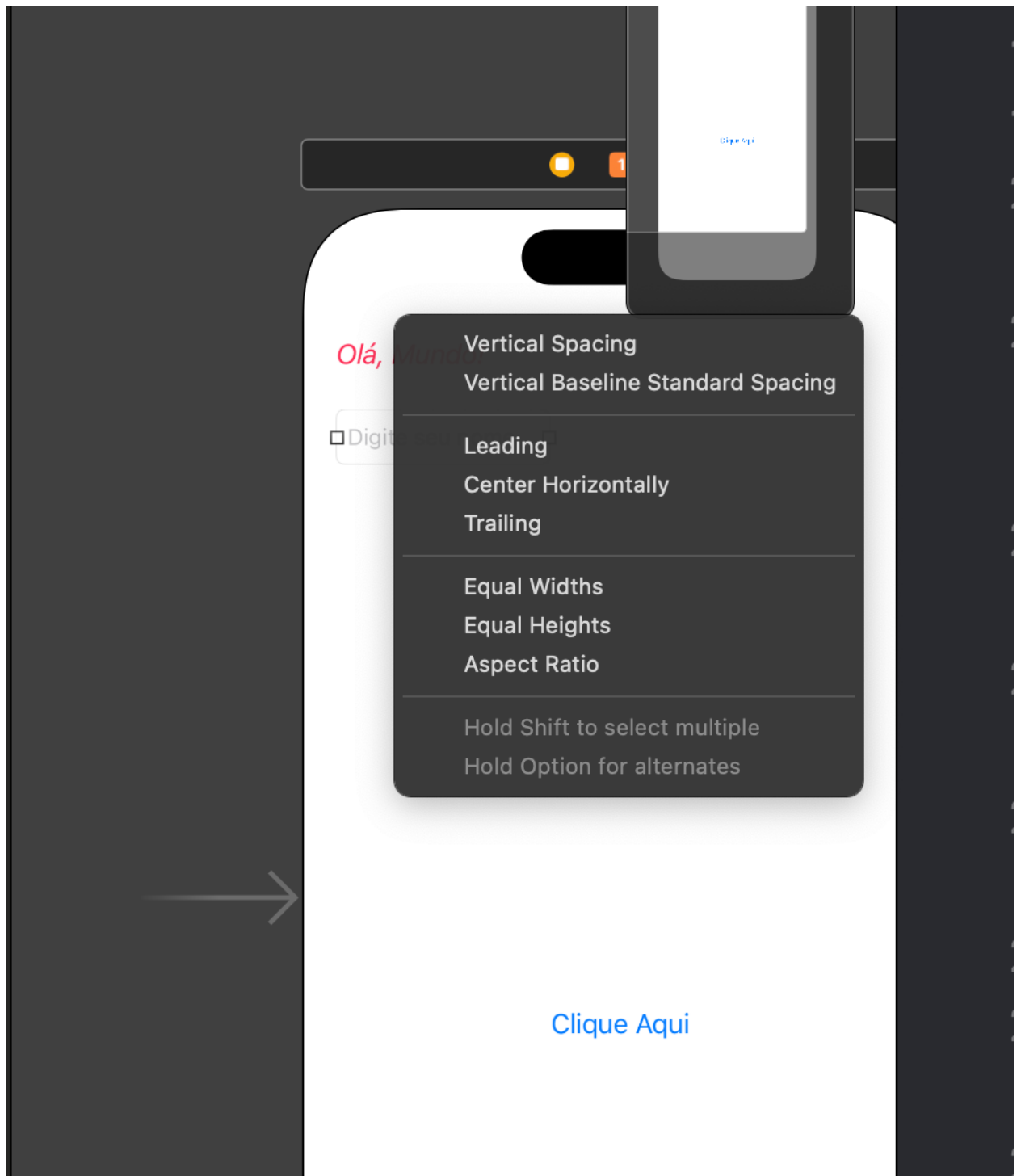
Captura de Tela 2025-03-31 às 19.37.26 (2)



4. Para o UITextField, podemos posicionar mais ou menos o componente no local que queremos, e adicionar constraints de outra forma, selecionando o UITextField, e depois segurando o control e arrastando até o label, dessa forma ele vai abrir um menu. Vamos selecionar center horizontally, e vamos repetir o processo selecionando vertical spacing.





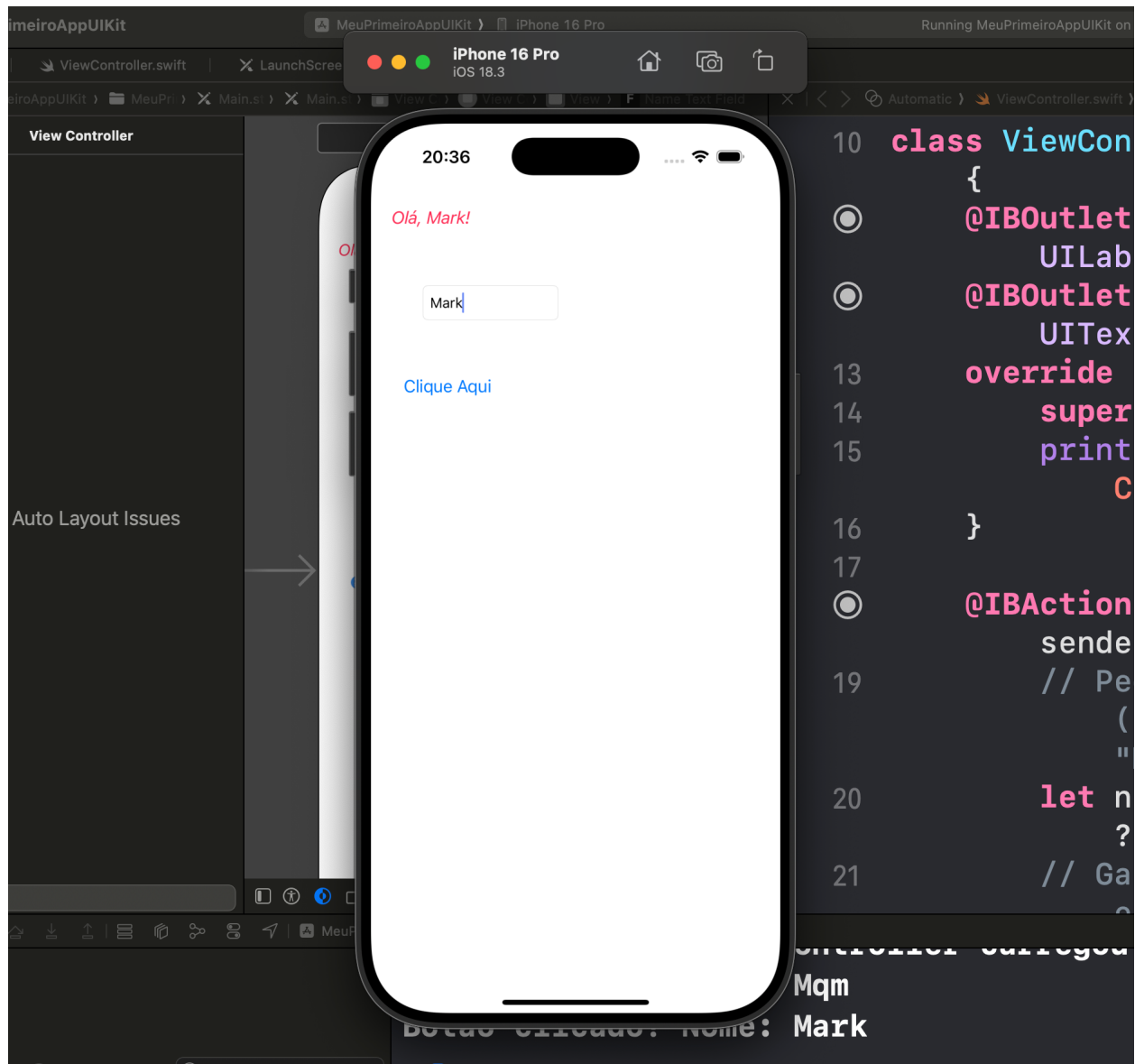


5. Faça o mesmo para o UIButton, adicionando constraints para a borda inicial (esquerda - 20) e para a **borda superior em relação ao TextField** (ex: 16 pontos abaixo do nameTextField).
  - *Nota do Professor:* Auto Layout pode ser complexo! Por agora, o objetivo é entender *por que* ele existe e experimentar adicionar algumas regras simples.

Linhas vermelhas/amarelas indicam problemas ou ambiguidades no layout. O Xcode pode sugerir correções.

### Passo 6: Testando no Simulador

- **O que é o Simulador?** Uma ferramenta do Xcode que imita um dispositivo iOS no seu Mac, permitindo testar seu app sem precisar de um iPhone/iPad físico (embora testar em dispositivos reais seja crucial eventualmente).
- **Atividade:**
  1. No topo da janela do Xcode, à direita do botão de "Play/Stop", selecione um Simulador (ex: "iPhone 15 Pro").
  2. Clique no botão "Run" (o ícone de Play .
  3. O Xcode compilará seu código e lançará o app no Simulador escolhido. Aguarde carregar.
  4. Interaja com seu app! Digite seu nome no campo de texto e clique no botão. Veja o label mudar!
  5. Verifique a Área de Debug no Xcode - você deve ver as mensagens que adicionamos com `print()`.
  6. Clique no botão "Stop" (o ícone de quadrado ) no Xcode para encerrar a execução.



Exercícios:

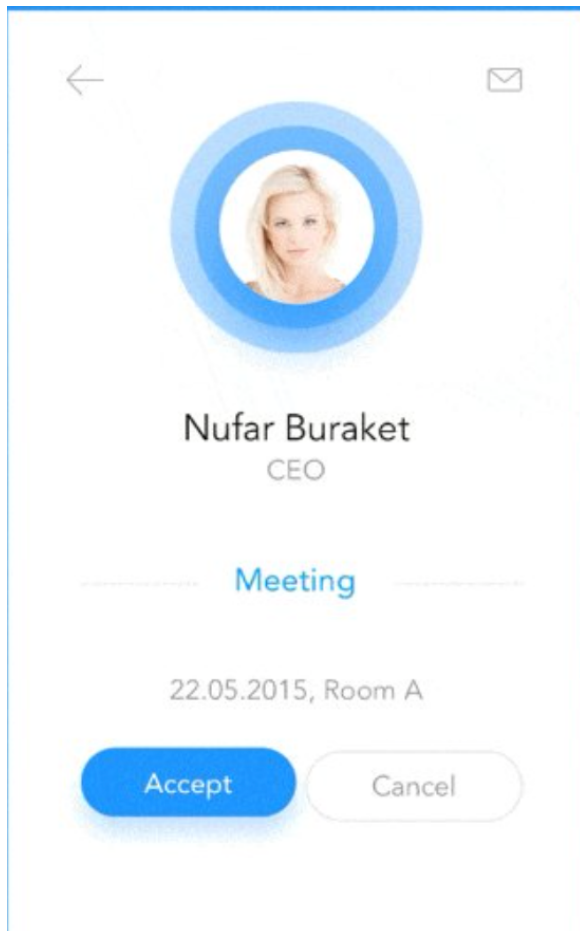
## 1. Autolayout

Aprofunde seus estudos de autolayout:

- <https://developer.apple.com/library/archive/documentation/UserExperience/Conceptual/AutolayoutPG/index.html>
- <https://medium.com/@luisfmachado/uiscrollview-autolayout-on-a-storyboard-a-step-by-step-guide-15bd67ee79e9>

- <https://www.youtube.com/watch?v=kK8l9iwz9kY>

Agora crie no storyboard uma tela parecida com:



## 2. Calculadora com Dois TextFields em UIKit (Swift)

Objetivo: Criar um aplicativo iOS simples utilizando UIKit e a linguagem Swift que funcione como uma calculadora básica, capaz de realizar operações aritméticas entre dois números inseridos pelo usuário.

Requisitos:

1. Crie um novo projeto Xcode do tipo "App" utilizando a linguagem Swift e a interface "Storyboard" ou "SwiftUI" (escolha uma e siga com ela).
2. Na sua **ViewController**, adicione os seguintes elementos de interface:
  - Dois campos de texto (**UITextField**) onde o usuário poderá inserir os dois números para o cálculo.

- Um rótulo (**UILabel**) para exibir o resultado da operação.
  - Quatro botões (**UIButton**), cada um representando uma operação aritmética básica:
    - Adição (+)
    - Subtração (-)
    - Multiplicação (×)
    - Divisão (÷)
3. Ao pressionar cada um dos botões de operação, o aplicativo deverá:
- Obter o texto dos dois **UITextField**s.
  - Converter esses textos para valores numéricos (você precisará lidar com possíveis erros de conversão, como quando o usuário não digita um número).
  - Realizar a operação aritmética correspondente ao botão pressionado.
  - Exibir o resultado no **UILabel**.
4. Certifique-se de que a interface do usuário seja organizada de forma clara e intuitiva.

#### Desafio Opcional:

- Adicione um botão "Limpar" que apague o conteúdo dos **UITextField**s e do **UILabel**.
- Implemente tratamento de erro para a operação de divisão por zero.